

crux

Turning raw test results into engineering decisions.

Project documentation — Gate 0: ground truth before product code.

Generated	2026-09-01
Commit	4958bbe feat: Gate 0 — corpus infrastructure, adapters, and the fingerprinting engine
Repository	github.com/adithyakm13/crux (private)
Licence	Apache-2.0
Tests	114 passing
Corpus	16 runs, 74 failures, 9 repositories
Gate 0	NOT MET

Every number in this document is computed from the code and the corpus at the commit above. None is estimated, rounded in the project's favour, or carried over from an earlier draft. Where a measurement has not been made, the document says so rather than supplying a plausible figure.

1 What crux is

A developer opens CI. 22 tests are red. Today they spend 40 minutes discovering it was one broken payment endpoint. crux is being built to tell them in four seconds: 18 failures share one cause, 3 share another, 1 is unrelated — and for each, the evidence.

The product extends along two axes. **Depth**: from grouping failures, to naming their cause, to predicting which changes are risky, to selecting which tests to run at all. **History**: from single-run analysis that works immediately with zero setup, to longitudinal intelligence that needs weeks of data but is where the compounding value lives. The single-run path must always work standalone; it is the adoption path, and every historical feature degrades to it gracefully.

1.1 The core distinction

A test failure is not a bug. crux exists to separate these cases, because each implies a different action:

Class	Means	Action
PRODUCT_REGRESSION	Code under test broke	Fix the product
TEST_DEFECT	Test is wrong	Fix the test
FLAKY	Nondeterministic	Quarantine, then fix
ENVIRONMENT_FAILURE	Infrastructure broke	Fix CI, retry
DEPENDENCY_FAILURE	External service broke	Retry or stub
DATA_FAILURE	Fixture or state wrong	Fix data setup
PERFORMANCE_REGRESSION	Slower, not wrong	Investigate perf
UNKNOWN	Insufficient evidence	Say so

UNKNOWN is a first-class answer. A system that guesses to avoid saying UNKNOWN is worse than no system, because users calibrate on early wrong answers and never come back.

2 Gate 0 — the current state

Every claim crux makes is a classification claim. Without labelled data none of them are falsifiable, and the project becomes an exercise in plausible-sounding output. So the corpus is built first, and no product code ships before it.

Requirement	Observed	Required	
≥ 80 complete CI runs	16	80	FAIL
≥ 50 runs with ≥ 5 failures each	3	50	FAIL
≥ 1000 labeled failures total	0	1000	FAIL
≥ 10 distinct repositories	9	10	FAIL
≥ 3 frameworks	4	3	PASS
≥ 2 independent labelers	0	2	FAIL
synthetic runs $\leq 20\%$ of corpus	0	20	PASS

Gate 0: NOT MET. Frameworks present, each backed by evidence in the payload rather than asserted: junit-jvm, playwright, pytest, vitest. A further 5 failures come from a framework that could not be identified and are reported separately rather than folded into the count.

2.1 What tooling cannot produce

Two Gate 0 numbers require people. They are named explicitly in the tool's own output so they cannot be quietly skipped:

- **Inter-labeler agreement.** Two people label the same 100-failure subset independently. Cohen's kappa on category, pairwise agreement on grouping. Below 0.80 pairwise agreement the task is ambiguous as specified, and every downstream target needs revisiting before it is chased. Human agreement is the ceiling: a model scoring above it is overfitting the labeler.
- **Separability.** One labeler sees *only* the failure payload — message, stack, stdout, timing, exit code. No repository, no git, no logs. Agreement with the full-context label over 150 failures *is* the separability rate. If its Wilson upper bound is below 60%, the payload-only thesis is wrong and crux stops.

The tooling enforces the second condition rather than trusting the labeler to self-police: in payload-only mode the repository, workflow, branch, commit and run URL are withheld from what is rendered, and the separability command refuses to score a labeler whose stored context is not payload-only. Both behaviours are asserted by test.

2.2 How every gate is evaluated

- Point estimates alone are not gate evidence. Every metric carries a 95% interval — Wilson for proportions, bootstrap for clustering scores.
- Bootstrap resamples **runs**, never pairs. Pairs within a run are strongly correlated; treating them as independent produces intervals several times too narrow, which turns a failing gate into a passing-looking number.
- A gate passes when the **lower** bound clears the threshold. A point estimate of 0.82 with a lower bound of 0.68 has not passed a 0.80 gate; it has produced a number.
- Held out means held out from the start. No threshold is tuned on data it is later evaluated on.

These are implemented in @cruxci/core as `wilson`, `bootstrapPairF1`, `gatePasses`, `cohensKappa` and `pairwiseAgreement`, so they apply by construction rather than by remembering to.

3 What has been built

Package	Purpose	Src	Test	External deps
adapter-junit	hardened streaming JUnit XML adapter	636	349	saxes
adapter-playwright	Playwright JSON and blob-report adapter	548	0	none
core	framework-neutral domain model and versioning for test failure analysis	826	222	none
corpus-tools	corpus harvesting, labeling and gate measurement	2072	260	fflate
engine	normalization, fingerprinting, clustering and classification	908	467	@noble/hashes

Line counts are of TypeScript source and tests. The dependency column is the whole external surface: three libraries across the project. Every one was added against a stated tradeoff rather than by reflex.

3.1 Not built, deliberately

Clustering, classification, the crux CLI, storage, history, flakiness, risk, selection, the dashboard and AI do not exist. They are Phase 1 and later, and Phase 1 does not begin until Gate 0 is met or is reported as failed. Building them now would mean tuning them against no ground truth, which is the failure mode Gate 0 exists to prevent.

3.2 Ingestion: input is hostile

Test results come from CI, which runs code from pull requests. Every byte is treated as attacker-controlled:

- **DOCTYPE is rejected outright** in JUnit XML. That closes XXE and billion-laugh at the door rather than relying on a resolver hook being wired correctly. No legitimate JUnit producer needs one.
- Hard caps on total bytes, nesting depth, attributes per element, text per field, name length and total attempts. Each is a clear error, never an out-of-memory crash.
- No value parsed from input is ever used as a filesystem path or passed to a shell.
- Ingest is idempotent on (run id, shard, test identity, attempt index). A CI job that uploads twice must not double-count; duplicate ingest silently corrupts every statistic downstream.

3.3 The attempt is the grain

A test that runs three times with retries is three attempts, never collapsed. Retries are evidence, not noise to be hidden. This is visible in the corpus: one Ghost failure appears as three attempts of the same test, because Playwright retried it twice and both retries also failed.

4 Normalization and fingerprinting

Deterministic, offline, versioned. 25 rules, applied in a fixed order, each individually toggleable and individually tested so the ablation table can be produced by switching rules off rather than by editing code.

4.1 The strict/loose split

Two outputs per failure. **Strict** keeps "expected 5, got 7" distinct from "expected 5, got 9", because those are different bugs. **Loose** collapses them, because they are the same broken function. Clustering needs both, and the split is load-bearing rather than cosmetic.

```
strict = H( error_type ■ strict_norm(message) ■ top5(app_frames, with line) )
loose  = H( error_type ■ loose_norm(message)  ■ top3(app_frames, no line)  )
```

H is BLAKE3 with 128 bits retained. Identifiers carry the algorithm version — fp_v1_<hex32> — so a fingerprint from a different version can never be silently compared against one from this version.

On the collision gate: BLAKE3-128 will not collide at any scale this system reaches. A collision between two distinct labelled root causes is therefore always two causes reducing to the same normalized string — a normalization defect, investigated as an over-aggressive rule, never as a hashing problem.

4.2 Properties, asserted by test

- Normalizing twice equals normalizing once — for the full cascade and for every rule individually. A rule that is not idempotent on its own makes the cascade depend on rule order in a way nobody can reason about.
- Normalization never throws on any corpus input.
- The same failure fingerprints identically on Linux, macOS and Windows. Path separators and line endings are a real source of drift, and CI runs the engine tests on all three.

5 The corpus

Specified in **runs**, not failures. Clustering quality is only measurable within a run: a corpus of isolated failures yields zero pairs to score.

Sourced from public GitHub Actions runs that upload test artifacts. The harvester walks recent failed runs, downloads non-expired artifacts, dispatches on file *contents* rather than filename, parses with the real adapters, and keeps only runs that actually contain failures.

Repository	Runs	Failures	Licence
openobserve/openobserve	1	28	AGPL-3.0
TryGhost/Ghost	5	18	MIT
recharts/recharts	1	9	MIT
apache/beam	3	7	Apache-2.0
ag-grid/ag-grid	2	3	unidentified
solidtime-io/solidtime	1	3	AGPL-3.0
Milkdown/milkdown	1	2	MIT
PostHog/posthog	1	2	unidentified
trinodb/trino	1	2	Apache-2.0

Runs from repositories whose licence GitHub cannot identify are excluded from the committed corpus by default, matching the harvester's own policy. GitHub reports NOASSERTION for a LICENSE file it cannot classify, which is not the same as having no licence; the two are recorded and reported separately.

5.1 Artifact retention is the binding constraint

Across the repositories examined, **20 to 22 of every 25 failed runs had already expired**. Most active repositories keep Actions artifacts for one to a few days, so a single harvesting pass can only ever reach a recent window, regardless of how many repositories are in the list.

Consequence: reaching the Gate 0 run count is a scheduled, repeated operation rather than a one-off. The harvester is idempotent on run id, so repeated runs accumulate instead of duplicating.

5.2 Contamination guards on labelling

The corpus is the measurement instrument, so the labelling tool is built to avoid three specific ways of corrupting it:

- **Seeing the other labeler's answers.** Each labeler writes to their own directory and never reads another's. Agreement measured after one labeler saw the other's work is not agreement, it is copying.
- **Seeing crux's answer.** The tool never shows a predicted cluster or category. A labeler shown the model's guess anchors on it, and the resulting F1 measures agreement with the model rather than with the truth.
- **Seeing more than the condition allows.** Payload-only mode withholds repository, workflow, branch and commit, because that is the precise condition the separability rate is defined over.

6 Measured findings

6.1 The deepest-app-frame signal is mostly unavailable

The specification calls the deepest application frame "the single most stable and most causally meaningful signal in a stack trace" and assigns it a clustering weight of 0.60. Measured over the corpus:

Framework	Failures	With a frame	With app frame	Rate (Wilson 95%)
playwright	55	55	0	0.000 [0.000, 0.065]
pytest	9	9	5	0.556 [0.267, 0.811]
unidentified	5	5	0	0.000 [0.000, 0.434]
vitest	3	3	2	0.667 [0.208, 0.939]
junit-jvm	2	2	2	1.000 [0.342, 1.000]
total	74	74	9	0.122 [0.065, 0.215]

The split is the finding, not the total. Playwright end-to-end failures have **no** application frame at all: the stack's only frame is the spec file, which classifies as test code, because the failure happened inside Playwright's own machinery acting on behalf of the test. Unit-test frameworks are the opposite — pytest and vitest supply one more often than not.

So the 0.60 weight is not wrong, but the signal is unavailable for exactly the workload that produces the most failures per run. Clustering evaluation must therefore report per-framework F1; an aggregate would be dominated by the end-to-end case where the signal is missing entirely, and would say nothing about the case where it is present.

6.2 Defects that only real CI data exposed

Each of these passed hand-written fixtures and failed on real artifacts. They are recorded because the pattern — plausible code, wrong on real input — is the thing the corpus exists to catch.

- **Raw ESC is illegal in XML 1.0**, in raw form and as a character reference, yet Playwright, pytest and several Maven reporters write ANSI colour codes into JUnit XML anyway. A strict parser rejected files that real runners really produce. Whole escape sequences are now removed at the stream level, counted, and reported — removing only the ESC byte would leave [2m as literal text and defeat the ANSI rule downstream.
- **The loose assertion rule swallowed whole messages**. It reduced every pytest failure to `AssertionError: <val>`, which would have collapsed unrelated failures into one cluster and inflated false positives. Narrowed to recognised expected/actual constructs, with literal collapsing split into its own ablatable rule.
- **JVM frames carry the package in the function name**, not the file path, so checking only the path classified `org.junit.runners.ParentRunner` as application code — the exact frame that must read as framework noise.
- **Vitest emits frames marked with a heavy arrow** rather than `at`, so crux parsed zero frames from them and discarded a real application frame. Found only because a framework count looked suspiciously easy to pass.
- **A property test caught a chunk-boundary bug** in the ANSI sanitizer: an OSC escape split across a stream chunk was mis-detected as complete and shredded. The result now provably does not depend on

how the stream was chunked.

6.3 A correction

An earlier draft of this analysis reported the app-frame signal as absent from *every* failure, and reported three frameworks by counting `junit` as one. Both were wrong. The first was partly a missing Vitest parser rather than a property of the data; the second counted a *format* — emitted by `pytest`, `jest`, `vitest` and `surefire` alike — as though it identified a framework. Only an adapter whose name is genuinely framework-specific now counts as evidence, and the unidentified remainder is reported alongside rather than folded in.

7 Limitations

Maintained under a standing rule: if something cannot be implemented correctly yet, document the limitation, provide a safe fallback, and add a roadmap entry — never pretend. Each entry below names its fallback, so none is a silent failure. The full list lives in `docs/limitations.md`.

Limitation	Why	Safe fallback
Playwright JSON is parsed whole, not streamed	The blob report streams line by line, but the JSON reporter emits one document and crux has no incremental JSON parser. Capped at 256 MiB.	Explicit <code>SIZE_LIMIT</code> error naming the blob reporter, never an OOM.
JUnit cannot express retries	Surefire rerun elements are parsed when present, but a producer that cannot emit them looks identical to a run where no retry happened.	<code>capabilities().retries</code> is false, so downstream treats absence as uninformative rather than as evidence.
Duplicate testcases are treated as retries	Genuinely ambiguous: some producers mean a retry, others mean two parameterized cases sharing a label.	Both preserved as separate attempts, with a warning.
Frame ordering is inferred per dialect	Python and pytest print outermost-first; V8, JVM, Go and Ruby print innermost-first. A stack mixing dialects may be ordered wrongly.	Reversal triggers on a majority of Python-parsed frames.
Shard index is the artifact's position	The provider does not report which shard produced an artifact.	Claims only 'these came from different uploads'; no machine identity or clock-skew correction is derived from it.
Unidentified-licence repositories are skipped	GitHub reports NOASSERTION for a LICENSE it cannot classify. This excludes some of the richest JUnit XML sources found.	Reported separately from 'no licence', and overridable explicitly.

8 Phases and gates

Each phase has a numeric gate. The next phase does not begin with the current gate unmet — if a gate is missed, the number actually achieved is reported instead.

	Phase	Scope	Gate
0	Evidence current	Corpus and evidence document.	Separability $\geq 60\%$, inter-labeler agreement $\geq 80\%$.
1	Core, single-run	Parsing, normalization, fingerprinting, clustering, deterministic classification, analyze and explain. Git tiers 1-2.	Pairwise F1 ≥ 0.80 ; fingerprint stability $\geq 99\%$; zero normalization collisions; 10k results < 5 s; per-category precision ≥ 0.85 .
2	Distribution	GitHub Action, PR comment, Checks, npx install, cross-platform binaries.	Clean install on macOS, Linux and Windows on ARM64 and x64, verified in CI.
3	History	SQLite, rollups, test identity with alias review, flakiness engine.	Rename recall $\geq 90\%$ at precision $\geq 98\%$; flake precision ≥ 0.85 ; historical query < 1 s over 10M attempts.
4	Change intelligence	Git tiers 1-3, risk model, ownership, components, PostgreSQL.	Risk correlation published, or marked uncalibrated in the UI itself.
5	Selection and dashboard	Coverage mapping, learned correlation, test selection, dashboard.	Selection recall $\geq 95\%$ on a time-ordered forward replay across ≥ 200 runs.
6	Ecosystem	Plugin SDK, pytest, Cypress, AI providers, OpenTelemetry correlation.	—

8.1 Explicitly out of scope for Phase 1-2

Dashboard, server, AI, PostgreSQL, plugin SDK, flakiness, risk, selection, quality score, demo application, telemetry — regardless of how easy any of them looks.

9 Standing commitments

9.1 Privacy

No telemetry. Not opt-out, not anonymous, none. This is permanent and is stated in the README. Three clearly separated modes, and the CLI states which one it is in: local (nothing leaves the machine), server (data goes to your own deployment), AI (a specified payload goes to a configured provider). Everything works air-gapped except AI and the GitHub integration.

9.2 The AI boundary

crux is fully useful with AI disabled, and AI is off by default. It may only rephrase deterministic findings, rank existing hypotheses, propose fixes for a human to read, and summarise computed statistics. It may never invent a commit, log line, test result, metric, owner or fingerprint; every AI claim must reference an evidence id that resolves, and a claim whose ids do not resolve is dropped before display. AI never takes an action — no merging, closing, quarantining or suppressing — regardless of configuration.

9.3 No fabricated numbers

No placeholders, no mocks presented as working, no hard-coded demo output. Every number a user sees is computed or cited. No hand-written sample output ships, in the README or anywhere else — it will be

wrong, and it is the first thing a sceptical reader checks. This document is generated from the code and the corpus for the same reason.

10 Running it

Node 20+ and pnpm 10+. Node 25 no longer bundles corepack, so pnpm is installed directly.

```
npm i -g pnpm@10
pnpm install
pnpm build
pnpm test
```

Building and measuring the corpus:

```
gh auth login
pnpm corpus harvest --repos corpus/sources.txt --runs-per-repo 25
pnpm corpus status

pnpm corpus label --labeler alice --context full
pnpm corpus label --labeler bob --context payload-only

pnpm corpus agreement --a alice --b bob
pnpm corpus separability --full alice --payload bob
pnpm corpus baseline --labeler alice
```

10.1 The next measurement

The baseline spike is deliberately sequenced *before* the 1000-label investment. Label 50 failures across 8 multi-failure runs, cluster with the crudest possible rule — exact loose-fingerprint match, no graph, no weights — and measure pairwise F1. It costs two days and is the highest-information move available.

Naive F1	Conclusion
near 0.75	The weighted graph is over-engineering. Ship the simple thing.
near 0.30	The problem is genuinely hard, and we know before building the full corpus.
near 0.05	The payload does not carry the signal. Stop and reconsider.

The harness for this is implemented and tested; it has no labels to score against. Of the 16 runs harvested, 3 carry five or more failures, which is the class that contributes most of the pair signal.

10.2 Reading the exit codes

0 pass, 1 gate failed, **2 insufficient data to evaluate**, 3 execution error. Two is not one: failing a build because the tool lacked data is how tools get removed from pipelines.